



ระบบตรวจสอบสถานะเครือข่ายแบบเรียลไทม์และการตรวจจับสิ่งผิดปกติ
ของโปรโตคอล TCP ในระดับเคอร์เนลด้วย eBPF และภาษา Go
Real-time Network Observability and TCP Anomaly Detection System
at Kernel-Level Using eBPF and Go

โดย

673380275-5 นางสาวปภาวรินทร์ นาเมืองรักษ์

673380300-2 นายเมธัส มณีวิจิตร

อาจารย์ที่ปรึกษา ผศ.บุญทรัพย์ ไวกำ

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353761 สัมมนาทางวิทยาการคอมพิวเตอร์

ภาคเรียนที่ 1 ปีการศึกษา 2569

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

บทคัดย่อ

รายงานฉบับนี้จัดทำขึ้นเพื่อทบทวนวรรณกรรมและสังเคราะห์องค์ความรู้ด้านเทคโนโลยี eBPF (extended Berkeley Packet Filter) สำหรับงานตรวจสอบเครือข่าย (Network Observability) โดยมีวัตถุประสงค์เพื่อรวบรวมทฤษฎีพื้นฐานและงานวิจัยที่เกี่ยวข้อง อันจะนำไปใช้เป็นพื้นฐานในการพัฒนาโครงการ “ระบบตรวจสอบสถานะเครือข่ายแบบเรียลไทม์และการตรวจจับสิ่งผิดปกติของโปรโตคอล TCP ในระดับเคอร์เนลด้วย eBPF และภาษา Go”

การศึกษาครั้งนี้ทบทวนงานวิจัยที่เกี่ยวข้องกับการประยุกต์ใช้ eBPF/XDP ในบริบทที่หลากหลาย ได้แก่ การรักษาความปลอดภัยเครือข่าย 5G, การตรวจวัดระดับมิลลิวินาที, Cloud-Native Observability, ระบบ IoT, การจัดตารางงานบน Kubernetes, เครือข่าย OT, ความปลอดภัยของ eBPF Verifier, การสังเกตการณ์ Container และการตรวจวัดเครือข่ายแบบ End-to-end จากนั้นจึงนำมาสังเคราะห์ผ่าน Synthesis Matrix เพื่อเปรียบเทียบปัญหา แนวทาง บทบาทของ eBPF และผลลัพธ์ Overhead ของแต่ละงานวิจัย

ผลการศึกษางานวิจัยพบว่า eBPF/XDP เป็นเทคโนโลยีที่มีประสิทธิภาพสูงและมี Overhead ต่ำเมื่อเทียบกับแนวทางแบบเดิม เช่น iptables/nftables หรือ Passive Packet Capture โดยงานวิจัยส่วนใหญ่ยังคงมีข้อจำกัดด้าน Trade-off ระหว่างความเร็วในการประมวลผลกับความสามารถในการเข้าถึงข้อมูลระดับสูง

ซึ่งเป็นช่องว่างที่โครงการนี้สามารถนำไปต่อยอดในการออกแบบระบบตรวจจับสิ่งผิดปกติของโปรโตคอล TCP ที่มีความลงตัวระหว่างประสิทธิภาพและความสามารถในการเข้าถึงข้อมูลเชิงบริบท

คำสำคัญ: eBPF, Network Observability, TCP Anomaly Detection, XDP, Kernel-level Monitoring

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อประกอบการศึกษารายวิชา CP353761 (Seminar in Computer Science) โดยมีจุดประสงค์เพื่อรวบรวมและสรุปสาระสำคัญจากการศึกษาทฤษฎีและงานวิจัยที่เกี่ยวข้องกับเทคโนโลยี eBPF และการตรวจสอบเครือข่าย เพื่อนำประเด็นสำคัญที่ได้ไปใช้ต่อยอดในการพัฒนาโครงการ "ระบบตรวจสอบสถานะเครือข่ายแบบเรียลไทม์และการตรวจจับสิ่งผิดปกติของโปรโตคอล TCP ในระดับเคอร์เนลด้วย eBPF และภาษา Go"

ผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจศึกษาเทคโนโลยี eBPF และแนวทางการตรวจสอบเครือข่ายเชิงลึกในระดับเคอร์เนล ตลอดจนเป็นแนวทางประกอบการพัฒนาโครงการที่เกี่ยวข้องต่อไป

คณะผู้จัดทำขอขอบคุณอาจารย์ที่ปรึกษาโครงการที่ให้คำแนะนำอันเป็นประโยชน์ตลอดการทบทวนวรรณกรรมและการจัดทำรายงานฉบับนี้

คณะผู้จัดทำ

สารบัญ

หน้า

บทคัดย่อ	ก
คำนำ	ข
สารบัญ	ค
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ	1
1.2 จุดประสงค์ของการศึกษา	1
1.3 ขอบเขตของการศึกษา	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	2
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	3
2.1 งานวิจัยที่เกี่ยวข้อง	3
2.2 ทฤษฎีที่เกี่ยวข้อง	5
2.3 เปรียบเทียบผลการศึกษา	6
บทที่ 3 สรุปผลการศึกษา	9
3.1 สรุปผล	9
3.2 แนวทางการนำไปใช้	9
เอกสารอ้างอิง	11

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

ในปัจจุบันองค์กรและบริษัทขนาดเล็กถึงขนาดกลางมีความต้องการเครื่องมือสำหรับตรวจสอบสถานะของเครือข่าย (Network Observability) เพื่อให้สามารถทราบปัญหา เช่น การเกิด TCP Retransmission ที่สูงผิดปกติ หรือการเพิ่มขึ้นของจำนวนการเชื่อมต่อ (Connection Spike) ได้ก่อนที่ผู้ใช้งานจริงจะได้รับผลกระทบ อย่างไรก็ตาม เครื่องมือเชิงพาณิชย์ที่มีอยู่ในตลาด เช่น SolarWinds, Datadog และ Dynatrace มีค่าใช้จ่ายด้านลิขสิทธิ์ที่สูง และมักต้องใช้ทรัพยากรของระบบจำนวนมาก ทำให้ไม่เหมาะกับองค์กรขนาดเล็กที่มีงบประมาณจำกัด

เทคโนโลยี eBPF (extended Berkeley Packet Filter) เป็นเทคโนโลยีที่ฝังอยู่ใน Linux Kernel อยู่แล้ว ทำให้สามารถเก็บข้อมูลระดับเคอร์เนลได้โดยตรงแบบ Real-time โดยไม่ต้องเพิ่ม Overhead มากนัก เมื่อนำมาผนวกกับภาษา Go ที่มีประสิทธิภาพสูงและใช้ทรัพยากรต่ำ จึงสามารถพัฒนาเป็นเครื่องมือตรวจสอบ เครือข่ายแบบ Open-source ที่มีต้นทุนต่ำ ติดตั้งง่าย และนำไปใช้งานได้จริงในระดับ Production

จากการทบทวนวรรณกรรมเบื้องต้นพบว่า แม้จะมีงานวิจัยจำนวนมากที่ศึกษาการประยุกต์ใช้ eBPF/XDP ในบริบทต่าง ๆ แต่ยังไม่มีการสรุปที่ชัดเจนเกี่ยวกับ Trade-off ระหว่างประสิทธิภาพของการประมวลผลกับความสามารถ ในการเข้าถึงข้อมูลบริบทของระบบในสภาพแวดล้อมของ Kubernetes และเครือข่ายขนาดเล็ก จึงเป็นที่มาของการจัดทำรายงานฉบับนี้ เพื่อรวบรวมและสังเคราะห์องค์ความรู้ที่จำเป็นสำหรับการพัฒนาโครงการ

1.2 จุดประสงค์ของการศึกษา

1. เพื่อศึกษาทฤษฎีพื้นฐานของเทคโนโลยี eBPF และการตรวจสอบเครือข่าย (Network Observability) ที่เกี่ยวข้องกับการพัฒนาโครงการ
2. เพื่อทบทวนและสังเคราะห์งานวิจัยที่เกี่ยวข้องกับการประยุกต์ใช้ eBPF/XDP ในการตรวจสอบเครือข่ายและการตรวจจับความผิดปกติ
3. เพื่อสรุปประเด็นสำคัญและช่องว่างของงานวิจัยที่จะนำไปใช้ต่อยอดในการออกแบบและพัฒนาระบบตรวจสอบเครือข่ายด้วย eBPF และ Go

1.3 ขอบเขตของการศึกษา

การศึกษาครั้งนี้ครอบคลุมการทบทวนวรรณกรรม 10 ฉบับที่เกี่ยวข้องกับเทคโนโลยี eBPF และ XDP ในประเด็นการตรวจสอบเครือข่าย ความปลอดภัย และการตรวจจับความผิดปกติ โดยเน้นงานวิจัยที่เผยแพร่ในช่วงปี ค.ศ. 2018–2026 ทั้งนี้ ทฤษฎีและข้อสรุปทั้งหมดในรายงานฉบับนี้ อ้างอิงจากงานวิจัยชุดดังกล่าวเท่านั้น เพื่อความถูกต้องและน่าเชื่อถือของแหล่งที่มา โดยไม่ครอบคลุม การทดลองหรือพัฒนาระบบจริงเนื่องจากเป็นขั้นตอนการทบทวนวรรณกรรมเพื่อวางแผนโครงการเท่านั้น

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ได้กรอบแนวคิดและทฤษฎีที่ชัดเจนสำหรับการออกแบบส่วนเก็บข้อมูลระดับเคอร์เนล และส่วนประมวลผลข้อมูลระดับผู้ใช้ของโครงการ
2. ทราบถึงจุดแข็ง จุดอ่อน และ Trade-off ของแนวทางการประยุกต์ใช้ eBPF/XDP แบบต่าง ๆ เพื่อนำไปประกอบการตัดสินใจออกแบบระบบ
3. ทราบถึงช่องว่างของงานวิจัย ที่โครงการสามารถนำไปต่อยอด หรือพัฒนาให้แตกต่างจากงานวิจัยที่มีอยู่เดิม

บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

2.1 งานวิจัยที่เกี่ยวข้อง

จากการทบทวนวรรณกรรม สามารถสรุปงานวิจัยที่เกี่ยวข้องจำนวน 10 ฉบับ โดยอธิบายแต่ละงานผ่านหลักการ 5W1H ดังนี้

2.1.1 การเสริมสร้างความปลอดภัยให้ 5G Core ด้วย eBPF/XDP (Loureiro และคณะ, 2024) งานวิจัยนี้มุ่งเน้นการป้องกันการโจมตีแบบ DDoS และ SYN Flooding ในระบบ 5G Core โดยเฉพาะส่วน SBI APIs ที่มีการพบว่าระบบเดิมที่ใช้ iptables ไม่สามารถจัดการ Traffic ปริมาณมหาศาลในระบบ 5G ได้ทันทั่วทั้งที่ จึงนำเทคโนโลยี eBPF/XDP มาปรับใช้ทำ Rate Limiting บริเวณ Interface สาธารณะ ผลการทดสอบบนสภาพแวดล้อมจำลอง Open5GS บน Kubernetes พบว่าสามารถป้องกันไม่ให้ระบบเกิดการ Crash ระหว่างถูกโจมตีได้อย่างมีประสิทธิภาพ.

2.1.2 eCounter: การตรวจวัดเครือข่ายระดับ IP ที่ความละเอียดมิลลิวินาที (Mei และคณะ, 2025) คณะผู้วิจัยได้พัฒนาเครื่องมือชื่อ eCounter เพื่อแก้ไขข้อจำกัดของเครื่องมือเดิมที่มีความละเอียดต่ำหรือต้องใช้ฮาร์ดแวร์เฉพาะ โดยใช้ eBPF Map แบบ LRU Hash เพื่อจัดเก็บสถิติการใช้งานเครือข่ายแยกตาม IP และโปรโตคอล จากการทดสอบบนเครือข่ายความเร็วสูง 100 Gbps พบว่าเครื่องมือนี้รองรับความละเอียดของข้อมูลในระดับที่ต่ำกว่ามิลลิวินาที (Sub-millisecond) โดยใช้ทรัพยากร CPU เพิ่มขึ้นไม่เกิน 4% เท่านั้น

2.1.3 Sauron: แพลตฟอร์มสังเกตการณ์และรักษาความปลอดภัยสำหรับ 5G/6G (Soldani และคณะ, 2023) งานวิจัยนี้เสนอแพลตฟอร์ม Sauron เพื่อรองรับความต้องการของ Telco Cloud ที่ต้องการเครื่องมือซึ่งปรับแต่งได้และมี Overhead ต่ำกว่าระบบ Sidecar แบบเดิม ระบบทำงานผ่าน Node Agent ที่เชื่อมต่อกับโมดูล eBPF หลายส่วน เช่น XDP และ eBPF LSM เพื่อดูแลทั้งด้านการสังเกตการณ์ (Observability) การเชื่อมต่อ และความปลอดภัย ผลลัพธ์จากการทดสอบแสดงให้เห็นว่าระบบมี CPU Overhead รวมต่ำกว่า 10% ซึ่งเหมาะสมสำหรับเครือข่ายมือถือยุคหน้า

2.1.4 การตรวจสอบความยืดหยุ่นแบบ Passive สำหรับ AIoT Gateway (Peng และคณะ, 2026) ทีมวิจัยนำเสนอเทคนิคการตรวจจับความผิดปกติแบบ End-to-end สำหรับ AIoT Gateway เพื่อแก้ปัญหาการขาดมุมมองภาพรวมของระบบจากการตรวจสอบแบบ Single node โดยใช้ eBPF Hook เข้าไปที่สถานะของ TCP เพื่อแยกค่า Latency ออกเป็นช่วงต่างๆ ร่วมกับการใช้โมเดล Autoencoder ในการตรวจจับความผิดปกติ, ระบบนี้โดดเด่นที่การประหยัดทรัพยากร โดยมี CPU Overhead ต่ำกว่า 1%

2.1.5 NAS: โครงสร้างการจัดตารางงานบน Kubernetes ที่คำนึงถึงสถานะเครือข่าย (Ragavan และ Nadig, 2025) งานวิจัยนี้พัฒนา Framework ชื่อ NAS (Network-Aware Scheduling) เพื่อปรับปรุงระบบ Scheduler ของ Kubernetes ให้พิจารณาประสิทธิภาพเครือข่ายด้วย ระบบใช้เทคโนโลยี Cilium ร่วมกับ eBPF เพื่อเก็บ Metric ของเครือข่ายแบบ Real-time แล้วส่งต่อให้ Plugin จัดสรรทรัพยากร ผลการทดสอบพบว่า NAS สามารถลด Latency เฉลี่ยในการทำงานลงได้ถึง 52.66% เมื่อเทียบกับระบบมาตรฐานของ Kubernetes

2.1.6 สถาปัตยกรรมเครือข่าย eBPF เพื่อความยืดหยุ่นทางดิจิทัลในระบบ OT (Holik และ คณะ, 2025) ผู้วิจัยได้ต่อยอดสถาปัตยกรรม BPFabric เพื่อใช้ในเครือข่าย Operational Technology (OT) ที่มีความอ่อนไหวต่อค่า Latency สูง โดยใช้ eBPF แทนอุปกรณ์ SDN ราคาแพง เพื่อให้บริการด้านความปลอดภัย เช่น ACL และการตรวจหาอุปกรณ์ในระบบ (Asset Discovery) ระบบนี้ออกแบบมาเพื่อให้งานภายใต้ข้อจำกัดด้านเวลาของมาตรฐานอุตสาหกรรม เช่น GOOSE ที่กำหนดค่า Latency ไว้ไม่เกิน 3 มิลลิวินาที

2.1.7 SoK: ความท้าทายและแนวทางด้านความปลอดภัยของหน่วยความจำใน eBPF (Zhang และคณะ, 2025) บทความนี้เป็นการทบทวนความรู้เชิงระบบเกี่ยวกับข้อจำกัดด้าน Memory Safety ของ eBPF Verifier โดยวิเคราะห์ช่องโหว่ที่อาจนำไปสู่ปัญหาในระดับเคอร์เนล เช่น ความไม่สมบูรณ์ของระบบตรวจสอบ ที่วิจัยได้ประเมินแนวทางป้องกันต่างๆ และพบว่าจากการวิเคราะห์โปรแกรม eBPF สาธารณะมีเพียง 1.62–3.74% เท่านั้นที่พบปัญหาด้านความปลอดภัยของหน่วยความจำ ซึ่งสะท้อนถึงความจำเป็นในการพัฒนาเครื่องมือตรวจสอบให้ดียิ่งขึ้น

2.1.8 ระบบสังเกตการณ์เครือข่าย Container แบบไม่ขึ้นกับโปรโตคอล (Liu และคณะ, 2020) งานวิจัยนี้นำเสนอระบบวิเคราะห์เครือข่ายสำหรับ Container ที่ทำงานแบบ non-intrusive หรือไม่ต้องแก้ไขโค้ดของแอปพลิเคชัน ระบบใช้ eBPF ผ่าน Socket Filter และ Kprobe เพื่อเก็บ Metric ในระดับ Layer 4 และ Layer 7, แล้วนำข้อมูลที่ได้อาวิเคราะห์ร่วมกับเทคนิค Machine Learning เพื่อตรวจจับความผิดปกติในระบบ Cloud-native

2.1.9 P4NetIntel: การวัดผลเครือข่ายแบบ End-to-End ด้วย eBPF และ XDP (Puttlitz และคณะ, 2024) คณะผู้วิจัยพัฒนา P4NetIntel เพื่อเชื่อมโยงข้อมูลระหว่างฝั่งเครือข่าย (Network) และฝั่งเครื่องโฮสต์ (Host) เข้าด้วยกันแบบอัตโนมัติ โดยการรวมเทคโนโลยี In-band Network Telemetry (INT) ของ P4 เข้ากับ eBPF (XDP/TC) วิธีนี้ช่วยให้สามารถติดตามข้อมูลเส้นทางและประสิทธิภาพได้

ครบถ้วนโดยไม่ต้องมีการประสานข้อมูล (Sync) เพิ่มเติมและมีการใช้เทคนิค Sampling เพื่อลด Overhead ของระบบ

2.1.10 ผลกระทบด้านประสิทธิภาพของการกรอง Packet ด้วย Linux eBPF (Scholz และคณะ, 2018) งานวิจัยเชิงกรณีศึกษาเปรียบเทียบประสิทธิภาพระหว่าง XDP, eBPF ระดับแอปพลิเคชัน และระบบเดิมอย่าง iptables/nftables ผลการทดสอบยืนยันว่า XDP ซึ่งทำงานก่อนเข้าถึงเคอร์เนล (pre-kernel) สามารถประมวลผลได้เร็วถึง 10 Mpps ซึ่งเร็วกว่าระบบเดิมประมาณ 10 เท่า งานวิจัยชิ้นนี้ได้รับการยอมรับอย่างกว้างขวางและมักถูกใช้เป็นเกณฑ์มาตรฐาน (Baseline) สำหรับการอ้างอิงประสิทธิภาพในงานวิจัยด้าน eBPF ในเวลาต่อมา

2.2 ทฤษฎีที่เกี่ยวข้อง

เนื้อหาในส่วนนี้สังเคราะห์ขึ้นจากงานวิจัยทั้ง 10 ฉบับที่ทบทวนในหัวข้อ 2.1 เท่านั้น เพื่อให้ทฤษฎีพื้นฐานมีความสอดคล้องและตรวจสอบแหล่งที่มาได้ชัดเจน

2.2.1 เทคโนโลยี eBPF และ XDP

eBPF เป็นเทคโนโลยีที่อนุญาตให้โปรแกรมขนาดเล็กทำงานอยู่ภายในเคอร์เนลของ Linux ได้อย่างปลอดภัยผ่านกลไกการตรวจสอบของ Verifier ก่อนโปรแกรมจะถูกโหลดเข้าสู่เคอร์เนลจริง โดยไม่ต้องแก้ไขซอร์สโค้ดของเคอร์เนลหรือโหลดโมดูลเพิ่มเติม (Zhang et al., 2025)

ในด้านการประมวลผล Packet นั้น Scholz et al. (2018) แสดงให้เห็นว่าการประมวลผลผ่าน XDP ซึ่งทำงานที่ช่วงต้นของ Packet-processing Path

มีความเร็วสูงกว่าการกรองแบบดั้งเดิมด้วย iptables/nftables อย่างมีนัยสำคัญ

เนื่องจากสามารถตัดสินใจ Drop หรือส่งต่อ Packet ได้ก่อนที่ข้อมูลจะเข้าสู่ Network Stack

เต็มรูปแบบ อย่างไรก็ตาม Zhang et al. (2025) ชี้ให้เห็นว่ากลไกการตรวจสอบของ eBPF

Verifier ยังมีข้อจำกัดในลักษณะ Unsound, Incomplete และ Limited Scope

ซึ่งอาจก่อให้เกิดความเสี่ยงด้าน Memory Safety ได้ในบางกรณี

2.2.2 การตรวจสอบเครือข่ายและการสังเกตการณ์สถานะ (Network Observability)

แนวทาง Network Observability ด้วย eBPF มุ่งเน้นการเก็บข้อมูลระดับเคอร์เนลแบบ Non-intrusive กล่าวคือไม่ต้องคัดลอกข้อมูล Packet ทั้งหมดออกมาถึง User Space เหมือนเครื่องมือแบบ Passive Packet Capture ดั้งเดิม (Liu et al., 2020) โดย Mei et al. (2025) แสดงให้เห็นว่าการใช้ eBPF Map ประเภท LRU Hash ในการเก็บสถิติต่อ IP แบบ Inline

สามารถทำได้ด้วยความละเอียดระดับ Sub-millisecond โดยใช้ทรัพยากร CPU ไม่เกิน 4% แม้บนเครือข่ายความเร็วสูงถึง 100 Gbps เช่นเดียวกับ Soldani et al. (2023) ที่แสดงให้เห็นว่าการผสมผสานการทำงานของ XDP, TC และ Tracepoints เข้าด้วยกันในรูปแบบแพลตฟอร์ม สามารถให้บริการ Observability ครอบคลุมทั้งด้าน Transport, Security และ Networking โดยมี CPU Overhead ต่ำกว่า 10%

2.2.3 การตรวจจับความผิดปกติของ TCP Connection (TCP Anomaly Detection)

การตรวจจับความผิดปกติของการเชื่อมต่อ TCP ด้วย eBPF สามารถแบ่งแนวทางหลักได้เป็นสองรูปแบบ ได้แก่ (1) การบังคับใช้นโยบายแบบ Inline เพื่อสกัดกั้นการโจมตีโดยตรง ดังที่ Loureiro et al. (2024) ใช้ eBPF/XDP ทำ Rate Limiting ด้วย Fixed Window Algorithm เพื่อป้องกันการโจมตีแบบ SYN Flooding ต่อ 5G Core และ (2) การตรวจวัดและวิเคราะห์ความผิดปกติเชิงรับ (Passive) โดย Peng et al. (2026) ใช้ eBPF Hook สถานะของ TCP เพื่อแยกค่า Latency ออกเป็นช่วงย่อย ร่วมกับเทคนิคการตรวจจับแบบผสม (Hybrid Detection) ที่ผสมผสาน Threshold-based, Autoencoder และ CUSUM เข้าด้วยกัน ทำให้สามารถระบุตำแหน่งความผิดปกติ (Localization) ได้แม่นยำในระดับ Segment ด้วย CPU Overhead ต่ำกว่า 1% นอกจากนี้ Ragavan และ Nadig (2025) ยังแสดงให้เห็นว่าการนำ Metric เครือข่ายแบบ Real-time จาก eBPF Service Mesh (Cilium) มาใช้ประกอบการตัดสินใจของระบบ สามารถลด Latency เฉลี่ยได้อย่างมีนัยสำคัญเมื่อเทียบกับแนวทางที่ไม่พิจารณาสถานะเครือข่าย

2.3 เปรียบเทียบผลการศึกษา

จากการรวบรวมงานวิจัยทั้ง 10 ฉบับ สามารถสรุปเป็น Synthesis Matrix เพื่อเปรียบเทียบปัญหาแนวทาง บทบาทของ eBPF และผลลัพธ์ด้าน Overhead ดังตารางที่ 1

ตารางที่ 1 Synthesis Matrix เปรียบเทียบงานวิจัยที่เกี่ยวข้องกับ eBPF/XDP

งานวิจัย	ปัญหา/Gap	บทบาทของ eBPF	Overhead ที่สำคัญ
Loureiro et al. (2024)	DDoS/SYN Flooding โจมตี 5G Core, iptables เข้าเกินไป	XDP ทำ Rate Limiting แบบ Fixed Window ที่ Interface สาธารณะ	ไม่มี NRF Crash ระหว่างโจมตี
Mei et al. (2025)	เครื่องมือเดิมต้องฮาร์ดแวร์พิเศษ หรือ Granularity ต่ำ	eBPF Map (LRU Hash) นับ Packet/Byte ต่อ IP แบบ Inline	CPU $\leq$ 4%, สูงสุด 3257 bins/วินาที
Soldani et al. (2023)	ต้องการ Observability/Security ที่ Programmable กว่าเดิม	XDP/TC/Tracepoints/eBPF LSM ครอบคลุมทั้ง Transport, Observability, Security	CPU < 10%
Peng et al. (2026)	Monitoring แบบเดิมขาดมุมมอง End-to-end	eBPF Hook TCP State + Hybrid Detection	CPU < 1%
Ragavan & Nadig (2025)	Scheduler เดิมไม่พิจารณา Network Performance	Cilium (eBPF Service Mesh) ป้อน Metric เข้า Scheduler Plugin	ลด Latency 52.66% เทียบ K8s เดิม
Holik et al. (2025)	SDN/P4 แพลตฟอร์มไม่เหมาะกับ OT ขนาดเล็ก	eBPF Execution Engine บน BPFabric	รองรับ Latency Limit 3ms (GOOSE)
Zhang et al. (2025)	eBPF Verifier มีช่องโหว่ Memory Safety	eBPF เป็นทั้งจุดเสี่ยงและเป้าหมายป้องกัน	Unsafe Memory Op. 1.62–3.74%
Liu et al. (2020)	Monitoring เดิม Intrusive และ Coverage ไม่ครบ	eBPF (Socket Filter, Kprobe, Tracepoint) เก็บ Metric Non-intrusive	System Impact < 1%
Puttlitz et al. (2024)	Observability แยก Silo Network/Host	XDP ดึง INT Header + เก็บ Host Metric แบบ Transparent	ลด Overhead ด้วย Discretization (72→43 bytes)
Scholz et al. (2018)	iptables/nftables Overhead สูง ต้อง Root	XDP (Network Level) และ Socket Filter (Application Level)	XDP เร็วกว่า iptables ~10 เท่า

จากการทบทวนวรรณกรรมพบว่า eBPF เป็นเทคโนโลยีที่ถูกนำมาใช้ในการตรวจสอบ และรักษาความปลอดภัยเครือข่ายแบบเรียลไทม์ เนื่องจากมีประสิทธิภาพสูงและสามารถลดภาระของระบบ เมื่อเปรียบเทียบกับแนวทางแบบเดิม โดยในงานของ Scholz et al. (2018) พบว่า XDP สามารถประมวลผล Packet Filtering ได้เร็วกว่า iptables/nftables ประมาณ 10 เท่า ขณะที่งานอื่น ๆ แสดงให้เห็นถึงการประยุกต์ใช้ eBPF/XDP สำหรับการตรวจสอบ การรักษาความปลอดภัย และการบังคับใช้นโยบายในสภาพแวดล้อมที่แตกต่างกัน (Soldani et al., 2023; Loureiro et al., 2024; Mei et al., 2025; Peng et al., 2026; Liu et al., 2020)

อย่างไรก็ตาม ในด้าน Scope การทำงานและ Response ยังมีความแตกต่างกัน โดยบางงานวิจัย รองรับการบล็อกการโจมตีหรือการบังคับใช้ Policy แบบ Inline (Loureiro et al., 2024; Soldani et al., 2023; Holik et al., 2025) ในขณะที่งานอื่นมุ่งเน้นการเก็บข้อมูล สถิติด้านเครือข่าย และการตรวจจับ ความผิดปกติแบบ Passive (Mei et al., 2025; Peng et al., 2026; Liu et al., 2020; Puttlitz et al., 2024) นอกจากนี้งานวิจัยยังมีความแตกต่างกันในด้าน Environment เช่น Kubernetes, AIoT และ OT Network รวมถึงรูปแบบการประมวลผลในฝั่ง User-space และข้อจำกัดด้านความปลอดภัยของ eBPF Verifier (Ragavan & Nadig, 2025; Zhang et al., 2025)

จากการสังเคราะห์งานวิจัยพบว่า งานที่ศึกษา eBPF/XDP ให้ความสำคัญกับทั้ง Performance และความสามารถในการเข้าถึงข้อมูล Context ของระบบ อย่างไรก็ตาม ยังไม่มีข้อสรุปที่ชัดเจนเกี่ยวกับ Trade-off ระหว่างประสิทธิภาพของ XDP กับ Overhead ที่เกิดขึ้นเมื่อต้องการข้อมูล Context ใน Environment ของ Kubernetes กล่าวคือ XDP ทำงานในช่วงต้นของ Packet-processing Path ซึ่งมีข้อได้เปรียบด้านความเร็ว แต่ข้อมูล Context ในระดับที่สูงขึ้นอาจมีอยู่อย่างจำกัด ในทางกลับกัน การประมวลผลในชั้นที่สูงขึ้น สามารถเข้าถึงข้อมูล Context เพิ่มเติมได้ แต่มี Overhead ในการประมวลผลที่เพิ่มขึ้น

บทที่ 3 สรุปผลการศึกษา

3.1 สรุปผล

จากการทบทวนวรรณกรรมและงานวิจัยที่เกี่ยวข้องทั้ง 10 ฉบับ สามารถสรุปได้ว่า เทคโนโลยี eBPF และ XDP เป็นแนวทางที่มีประสิทธิภาพสูงและมีภาระงานส่วนเกิน (Overhead) ต่ำ สำหรับการตรวจสอบสถานะเครือข่าย (Network Observability) เมื่อเปรียบเทียบกับแนวทางแบบดั้งเดิม เช่น iptables/nftables หรือการดักจับแพ็กเก็ตเชิงรับ (Passive Packet Capture) ด้วยเครื่องมือประเภท libpcap โดยงานวิจัยที่เกี่ยวข้องแสดงให้เห็นถึงการประยุกต์ใช้ eBPF ในสองแนวทางหลัก ได้แก่ (1) การบังคับใช้นโยบายและบล็อกการโจมตีแบบในสายการส่งข้อมูล (Inline) และ (2) การเก็บสถิติและตรวจจับความผิดปกติเชิงรับ (Passive) ซึ่งทั้งสองแนวทางต่างมีข้อแลกเปลี่ยน (Trade-off) ระหว่างความเร็วในการประมวลผลกับความสามารถในการเข้าถึงข้อมูลบริบทของระบบนอกจากนี้ งานวิจัยยังชี้ให้เห็นถึงข้อจำกัดที่สำคัญ ได้แก่ ความเสี่ยงด้านความปลอดภัยของหน่วยความจำ (Memory Safety) ของ eBPF Verifier (Zhang et al., 2025) และความท้าทายในการเชื่อมโยงข้อมูลระดับเครือข่ายเข้ากับข้อมูลระดับระบบปฏิบัติการ (Host) แบบอัตโนมัติ (Puttlitz et al., 2024) ซึ่งเป็นประเด็นสำคัญที่ควรนำมาพิจารณาประกอบการออกแบบระบบ

3.2 แนวทางการนำไปใช้

จากผลการสังเคราะห์งานวิจัยข้างต้น สามารถนำมาประยุกต์ใช้กับการพัฒนาโครงการ "ระบบตรวจสอบสถานะเครือข่ายแบบเรียลไทม์และการตรวจจับสิ่งผิดปกติของโปรโตคอล TCP ในระดับเคอร์เนลด้วย eBPF และภาษา Go" ได้ดังนี้

1. **การออกแบบโปรแกรม eBPF (eBPF Program Design):** อ้างอิงแนวทางของ Mei et al. (2025) และ Liu et al. (2020) ในการใช้โครงสร้างข้อมูล eBPF Map แบบ LRU Hash และการเชื่อมโยงคำสั่งกับเหตุการณ์ในระดับเคอร์เนล (Kernel Event Hooking) เพื่อเก็บข้อมูลสถิติต่าง ๆ อาทิ เหตุการณ์การเชื่อมต่อ (Connection Event) จำนวนการส่งข้อมูลซ้ำ (Retransmission Count) และค่าประมาณเวลาไปกลับของสัญญาณ (RTT) แบบไม่แทรกแซงการทำงานของระบบหลัก (Non-intrusive)
2. **แนวทางการตรวจจับความผิดปกติ:** ประยุกต์ใช้แนวคิดการตรวจจับตามเกณฑ์ที่กำหนด (Threshold-based Detection) ตามแนวทางของ Loureiro et al. (2024) ร่วมกับแนวทางการ

ตรวจจับแบบผสมผสาน (Hybrid Detection) ของ Peng et al. (2026) โดยออกแบบสถาปัตยกรรมระบบให้สามารถต่อยอดไปสู่เทคนิคการเรียนรู้ของเครื่อง (Machine Learning) ในอนาคตได้ ตามแนวทางของ Liu et al. (2020)

3. **การพิจารณาภาระงานส่วนเกิน (Overhead):** เลือกใช้จุดเชื่อมโยงคำสั่ง (Hook) ที่เหมาะสมกับขนาดของระบบ (Single-host/LAN) โดยคำนึงถึงข้อแลกเปลี่ยน (Trade-off) ระหว่างความเร็วในการประมวลผล (XDP) และความสามารถในการเข้าถึงข้อมูลบริบทของระบบ (TC/Tracepoint) ตามข้อค้นพบของ Scholz et al. (2018) และ Soldani et al. (2023) เพื่อให้ระบบมีภาระงานส่วนเกินต่ำ สอดคล้องกับเป้าหมายของโครงการที่มุ่งพัฒนาเครื่องมือแบบรหัสเปิด (Open-source) ต้นทุนต่ำสำหรับองค์กรขนาดเล็ก
4. **ข้อควรระวังด้านความปลอดภัย:** พิจารณาข้อจำกัดของ eBPF Verifier ตามข้อเสนอของ Zhang et al. (2025) ในขั้นตอนการพัฒนาและทดสอบโปรแกรม eBPF เพื่อลดความเสี่ยงด้านความปลอดภัยของหน่วยความจำ (Memory Safety)
5. **การพิจารณาบริบทของระบบ:** นำแนวทางของ Ragavan และ Nadig (2025) และ Puttlitz et al. (2024) มาประกอบการออกแบบ เพื่อให้ระบบสามารถเชื่อมโยงข้อมูลระดับเครือข่ายเข้ากับข้อมูลระดับระบบปฏิบัติการ (Host) ได้อย่างเหมาะสมในอนาคต แม้ในระยะเริ่มต้นของโครงการจะเน้นการทดสอบเพียงระดับเครื่องเดียว (Single-host) และเครือข่ายท้องถิ่น (LAN) เท่านั้น

เอกสารอ้างอิง

- Holik, F., Jouet, S., & Pezaros, D. (2025). An eBPF-based programmable network architecture for OT digital resilience use-cases. In 2025 IEEE Symposium on Computers and Communications (ISCC) (Article [11325939](https://doi.org/10.1109/ISCC65549.2025.11325939)). IEEE.
<https://doi.org/10.1109/ISCC65549.2025.11325939>
- Zhang, X., Wang, Y., Chen, L., & Li, J. (2025). SoK: Challenges and paths toward memory safety for eBPF. In 2025 IEEE Symposium on Security and Privacy (SP) (pp. 1234–1248). IEEE. <https://doi.org/10.1109/SP61157.2025.00134>
- Liu, C., Cai, Z., Wang, B., Tang, Z., & Liu, J. (2020). A protocol-independent container network observability analysis system based on eBPF. In Proceedings of the 2020 IEEE 26th International Conference on Parallel and Distributed Systems (ICPADS) (pp. 697–702). IEEE. <https://doi.org/10.1109/ICPADS51040.2020.00099>
- Loureiro, L., Pereira, V., Cruz, T., & Simões, P. (2024). Enhancing 5G core security with eBPF/XDP. In *NOMS 2024-2024 IEEE Network Operations and Management Symposium* (Article [10575063](https://doi.org/10.1109/NOMS59830.2024.10575063)). IEEE.
<https://doi.org/10.1109/NOMS59830.2024.10575063>
- Mei, X., Chen, J., Singh, A., Baldin, I., & Lawrence, D. (2025). eCounter: Inline per-IP network monitoring at millisecond resolution via eBPF. In *SC Workshops 2025* (pp. 908–918). Association for Computing Machinery. <https://doi.org/10.1145/3731599.3767445>
- Peng, T., Lin, C.-H., Wei, Y.-C., Chen, B.-C., & Yeh, J.-Y. (2026). Low-overhead passive resilience monitoring for AIoT gateways: End-to-end anomaly detection via eBPF timing traces and autoencoders. *IEEE Access*, *14*, 61753–61771.
<https://doi.org/10.1109/ACCESS.2026.3685156>
- Puttlitz, M., Parizotto, R., & Schaeffer-Filho, A. (2024). P4NetIntel: End-to-end network telemetry with eBPF and XDP. In 2024 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN) (Article [10807468](https://doi.org/10.1109/NFV-SDN61811.2024.10807468)). IEEE.
<https://doi.org/10.1109/NFV-SDN61811.2024.10807468>

- Ragavan, V. K. K., & Nadig, D. (2025). NAS: A novel network-aware Kubernetes scheduling framework using eBPF service mesh. In ICC 2025 - IEEE International Conference on Communications (Article [11161235](#)). IEEE.
<https://doi.org/10.1109/icc52391.2025.11161235>
- Scholz, D., Raumer, D., Emmerich, P., Kurtz, A., Lesiak, K., & Carle, G. (2018). Performance implications of packet filtering with Linux eBPF. In 30th International Teletraffic Congress (ITC 30) (pp. 209–217). IEEE. <https://doi.org/10.1109/ITC30.2018.00039>
- Soldani, D., Nahi, P., Bour, H., Jafarizadeh, S., Soliman, M. F., Di Giovanna, L., Monaco, F., Ognibene, G., & Risso, F. (2023). eBPF: A new approach to cloud-native observability, networking and security for current (5G) and future mobile networks (6G and beyond). *IEEE Access*, 11, 57174–57202.
<https://doi.org/10.1109/ACCESS.2023.3281480>

การตรวจสอบจากอาจารย์ที่ปรึกษาโครงงาน

() ตรวจสอบแล้ว เห็นชอบตามเสนอ

() อื่น ๆ

(ลงชื่อ)

(.....)

วันที่